AeroFare

FLIGHT PRICE PREDICTION USING MACHINE LEARNING

A Project Report

submitted in fulfilment of the internship requirements at

Naviotech Solution Pvt. Ltd.

Domain: Data Science and Machine Learning

Submitted by

Team Member	Institution	University
Shaswat Bhatnagar	**Sagar Institute of Research and Technology**	RGPV
Kanishka Gaikwad	**Usha Mittal institute of technology**	SNDT
Javiya Chelsy	**B H.Gardi college of engineering and technology**	GTU

Participating institutions: Sagar Institute of Research and Technology (RGPV); Usha Mittal Institute of Technology (SNDT); B. H. Gardi College of Engineering and Technology (GTU)

Internship Mentor: Thomas Antony

Submission Date: 30 October 2026

Live Application: https://flight-fare-prediction-qb3btfdwzcnxafen8wttqj.streamlit.app/
GitHub Repository: https://github.com/chelsyjaviya130-svg/flight-fare-prediction

DECLARATION

We hereby declare that the project report titled **"AeroFare — Flight Price Prediction Using Machine Learning"** has been prepared by us as part of our internship project work at Naviotech Solution Pvt. Ltd. The report describes the project workflow, implementation, and evaluation results to the best of our knowledge. Any external datasets, libraries, and resources used have been acknowledged, and the work has not been submitted elsewhere for any other award.

Name	Date
Shaswat Bhatnagar	30 October 2026
Kanishka Gaikwad	30 October 2026
Javiya Chelsy	30 October 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Naviotech Solution Pvt. Ltd.** for providing us the opportunity to work on this project as part of our internship and for the guidance and learning environment it offered.

We thank our internship mentor **Thomas Antony**, for guidance and feedback during the project, for support during the preparation of this report.

We also thank our institutions — Sagar Institute of Research and Technology (RGPV), Usha Mittal Institute of Technology (SNDT), and B. H. Gardi College of Engineering and Technology (GTU) — for their encouragement.

We acknowledge the open-source Python ecosystem — including Pandas, NumPy, Scikit-learn, and Streamlit — and the publisher of the Kaggle dataset used in this project. Finally, we thank our team members and families for their cooperation and support.

Shaswat Bhatnagar | Kanishka Gaikwad | Javiya Chelsy

ABSTRACT

AeroFare is a machine-learning application that estimates flight ticket prices from journey details. The user provides information such as airline, origin and destination, travel class, date, departure and arrival times, duration, and number of stops. A trained Random Forest Regression model processes these features and returns an estimated fare through a Streamlit web interface.

The project covers data understanding, preprocessing, model training, evaluation, and cloud deployment. It uses the Freshly_cleaned.csv dataset of approximately 300,257 records and 25 columns, with ticket price as the target variable. The trained model and dataset are stored in a private Hugging Face repository, the source code is maintained on GitHub, and the application is deployed on Streamlit Community Cloud.

On the recorded test evaluation, the model achieved a Mean Absolute Error (MAE) of ₹1,434.21, a Root Mean Squared Error (RMSE) of ₹3,127.39, and an R^2 score of 0.9811. These values describe performance on the project's held-out test split and should not be interpreted as a guarantee of accuracy on future or live market fares. The dataset covers February and March observations only, so the application is a demonstration of an end-to-end workflow and does not fetch live airline prices.

Keywords: flight price prediction, regression, Random Forest, Streamlit, machine learning, deployment.

TABLE OF CONTENTS

1. Introduction..... 1
1.1 Introduction to Flight Price Prediction..... 1
1.2 Background and Motivation..... 1
1.3 Importance of Predictive Analytics in Aviation and Travel Planning..... 1
1.4 Problem Statement..... 1
1.5 Project Objectives..... 1
1.6 Project Scope..... 1
1.7 Proposed Solution..... 1
1.8 Expected Users and Practical Applications..... 1
2. Literature Review and Existing Systems..... 1
2.1 Overview of Flight-Fare Prediction Approaches..... 1
2.2 Traditional Fare Comparison and Booking Platforms..... 1
2.3 Limitations of Static Rules and Simple Price Estimates..... 1
2.4 Supervised Machine Learning for Regression..... 1
2.5 Existing Approach versus the Proposed Project..... 1
3. Dataset Description..... 1
3.1 Dataset Source and Overview..... 1
3.2 Target Variable..... 1
3.3 Features, Data Types, and Important Columns..... 1
3.4 Airlines, Routes, Dates, Class, Stops, and Duration..... 1
3.5 Missing-Value and Duplicate-Record Analysis..... 1
3.6 Data Preprocessing and Feature Preparation..... 1
3.7 Dataset Limitations and Coverage..... 1
4. Tools and Technologies..... 1
4.1 Technologies Used and Their Purpose in This Project..... 1
4.2 Software Requirements..... 1
4.3 Hardware Requirements..... 1
5. System Design and Methodology..... 1
5.1 System Architecture..... 1
5.2 End-to-End Workflow..... 1
5.3 Methodology Stages..... 1
5.4 User Prediction Workflow and Data Flow..... 1
6. Machine Learning Model..... 1
6.1 Why Flight-Price Prediction Is a Regression Task..... 1
6.2 Why Random Forest Regression Was Selected..... 1
6.3 Working Principle of Random Forest Regression..... 1

6.4 Input Features and Target Variable..... 1
6.5 Feature Encoding and Transformations..... 1
6.6 Training and Testing Methodology..... 1
6.7 Model Saving and Loading..... 1
6.8 Prediction Generation..... 1
6.9 Assumptions and Limitations..... 1
7. Evaluation and Results..... 1
7.1 Evaluation Methodology..... 1
7.2 Evaluation Metrics..... 1
7.3 Results..... 1
7.4 Interpretation of the Metrics for Fare Prediction..... 1
7.5 Evaluation Plots..... 1
7.6 Model Strengths..... 1
7.7 Potential Sources of Error..... 1
7.8 Generalization Limitations..... 1
8. Application Development..... 1
8.1 Overview of the Streamlit Application..... 1
8.2 User Inputs..... 1
8.3 Input Validation and Route Availability Check..... 1
8.4 Predicted Fare Display..... 1
8.5 Error Handling and User Guidance..... 1
9. Deployment and GitHub..... 1
9.1 Repository Organization..... 1
9.2 Dependency Management..... 1
9.3 Model-File Storage and Retrieval..... 1
9.4 Secrets Management..... 1
9.5 Deployment to Streamlit Community Cloud..... 1
9.6 Testing the Deployed Application..... 1
9.7 Maintenance and Troubleshooting..... 1
10. Conclusion and Future Scope..... 1
10.1 Conclusion..... 1
10.2 Learning Outcomes..... 1
10.3 Future Scope..... 1
References..... 1
Appendix A: Items to Verify Before Submission..... 1

If the Table of Contents appears empty, right-click it in Word and choose Update Field → Update entire table.

LIST OF FIGURES

No.	Title
Figure 3.1	Distribution of ticket price
Figure 3.2	Average ticket price by airline
Figure 3.3	Ticket price by travel class and number of stops
Figure 5.1	System architecture of AeroFare (editable diagram)
Figure 5.2	End-to-end machine-learning and deployment workflow (editable diagram)
Figure 5.3	Data flow of a single user prediction (editable diagram)
Figure 7.1	Actual versus predicted ticket price on the test set
Figure 7.2	Distribution of prediction errors (residuals)
Figure 7.3	Feature importance from the Random Forest model
Figure 8.1	AeroFare application homepage
Figure 8.2	Flight details input form
Figure 8.3	Validation or error message for an unsupported selection
Figure 8.4	Predicted fare result
Figure 9.1	GitHub repository structure

LIST OF TABLES

No.	Title
Table 2.1	Comparison of existing platforms and the AeroFare project
Table 3.1	Dataset summary
Table 3.2	Descriptive statistics of the target variable (price)
Table 3.3	Model input features
Table 4.1	Technologies used and their purpose
Table 4.2	Software requirements
Table 4.3	Recommended hardware requirements (not measured)
Table 6.1	Training configuration
Table 7.1	Recorded test-set evaluation results
Table 7.2	Error relative to recorded price statistics (derived, contextual)
Table 8.1	User inputs collected by the application
Table 9.1	Project links
Table 9.2	Deployment test checklist
Table 9.3	Common issues and remedies

LIST OF ABBREVIATIONS

Abbreviation	Full form
API	Application Programming Interface
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
HF	Hugging Face
INR (₹)	Indian Rupee
MAE	Mean Absolute Error
ML	Machine Learning
R^2	Coefficient of Determination
RMSE	Root Mean Squared Error
UI	User Interface
URL	Uniform Resource Locator

1. Introduction

1.1 Introduction to Flight Price Prediction

Air-ticket prices are not fixed. For the same pair of cities, the fare a passenger sees can change with the airline, the class of travel, the time of departure, the duration of the journey, the number of stops, and the date of travel. This variation makes it difficult for travellers to judge whether a quoted fare is reasonable and to decide when and how to travel.

Flight price prediction applies data-driven techniques to this problem. Instead of relying on manually written rules, a model learns the relationship between journey attributes and ticket price from historical records, and can then estimate the price for a new combination of attributes. Because the price is a continuous numerical value, the task is naturally framed as a supervised **regression** problem.

1.2 Background and Motivation

Airline pricing is influenced by many interacting factors such as demand, route popularity, class of service, and schedule. Travellers usually discover fares one search at a time on booking platforms, which show current offers but do not by themselves explain which journey attributes drive the price. A learning-based estimator can help users explore how price changes as the journey details change.

This project, **AeroFare**, was undertaken as a team internship project at Naviotech Solution Pvt. Ltd. in the domain of Data Science and Machine Learning. The motivation was to build a complete, practical machine-learning workflow: understanding a real-world dataset, preparing features, training and evaluating a model, and delivering the result to end users through a deployed web application.

1.3 Importance of Predictive Analytics in Aviation and Travel Planning

Predictive analytics converts historical data into forward-looking estimates. In aviation and travel planning, such estimates can support itinerary comparison, budgeting, and the understanding of how factors like travel class and number of stops relate to price. For students and early-career practitioners, flight-fare estimation is also a well-suited tabular regression problem because it combines categorical attributes (airline, route, class) with numerical attributes (duration, departure hour, stops).

1.4 Problem Statement

The problem addressed by this project is to estimate the price of a flight ticket from a set of journey attributes — airline, origin, destination, travel class, travel date, departure and arrival times, duration, and number of stops. A fixed-price or rule-based approach cannot

capture the patterns present in historical fare data. This project therefore trains a regression model on historical examples and exposes it through a web interface that returns a fare estimate for user-supplied journey details.

1.5 Project Objectives

- To explore and understand the Freshly_cleaned.csv flight-fare dataset, including its size, features, target variable, and data quality.
- To prepare numerical and categorical features suitable for model training.
- To train a Random Forest Regression model that estimates ticket price.
- To evaluate the model on a held-out test set using MAE, RMSE, and R^2.
- To build an interactive Streamlit application in which users enter journey details and view an estimated fare.
- To deploy the application online and maintain the source code in a GitHub repository with clear documentation.

1.6 Project Scope

The scope of the project covers the complete workflow from data understanding to cloud deployment. The application supports the airlines, routes, and travel classes present in the dataset, and returns an estimate when the selected route is supported by the data. The project does not retrieve live prices from airlines or booking platforms, and it does not make bookings. The dataset covers February and March only, so the estimates are indicative of patterns in that period.

1.7 Proposed Solution

AeroFare combines a trained Random Forest Regression model with a Streamlit web interface. The user selects journey details; the application validates the route, prepares the input in the feature format expected by the model, and obtains a predicted fare from the trained model. To keep the web application lightweight, the trained model and cleaned dataset are stored in a private Hugging Face repository and retrieved by the deployed application at run time using a token held in Streamlit Cloud Secrets.

1.8 Expected Users and Practical Applications

- **Travellers and students** who want a quick, indicative estimate of fares for different airlines, classes, and stop counts in the covered period.
- **Learners of data science** who want a worked example of a tabular regression workflow taken through deployment.
- **Developers** who may extend the project with additional data, models, or live-data integrations.

2. Literature Review and Existing Systems

2.1 Overview of Flight-Fare Prediction Approaches

Flight-fare prediction is commonly approached as a regression problem in which historical fare records are used to learn a mapping from journey attributes to price. Typical approaches described in the data-science literature include linear models, tree-based ensembles such as Random Forests and Gradient Boosting, and neural networks. Tree-based ensembles are widely used for structured, mixed-type tabular data because they can model non-linear relationships and interactions between features without extensive feature scaling.

A second family of approaches treats the problem as time-series forecasting of a specific route's price over the days leading up to departure. This requires repeated price observations of the same flights over time, which is not the structure of the dataset used in this project.

2.2 Traditional Fare Comparison and Booking Platforms

Existing travel platforms — airline websites, online travel agencies, and fare-comparison services — display the prices currently on offer for a chosen route and date. Their strength is that they show real, bookable fares. However, they are search tools: each query returns the prices available at that moment, and they do not by themselves expose a reusable model that explains how journey attributes such as class, number of stops, or departure time relate to price in historical data.

2.3 Limitations of Static Rules and Simple Price Estimates

- Fixed rules or average-price tables treat every journey on a route alike and cannot reflect combined effects of airline, class, duration, and stops.
- Hand-written rules are difficult to maintain as new routes or airlines are added.
- Simple averages cannot represent the strongly non-linear and skewed distribution of fares; in this dataset the mean price is far above the median, indicating a right-skewed distribution (see Chapter 3).

2.4 Supervised Machine Learning for Regression

In supervised regression, a model is trained on examples consisting of input features and a known numerical target. Training adjusts the model so that its predictions are close to the true targets, and a separate test set is used to measure how well it performs on unseen examples. For flight fares, the features are the journey attributes and the target is the ticket price. Ensemble methods such as the Random Forest (Breiman, 2001) combine many decision trees to reduce variance and improve predictive stability.

2.5 Existing Approach versus the Proposed Project

Table 2.1: *Comparison of existing platforms and the AeroFare project*

Aspect	Typical booking / comparison platform	AeroFare (this project)
Primary purpose	Show current, bookable fares	Estimate fare from historical patterns
Data used	Live airline inventory	Historical dataset (February–March)
Method	Search and listing	Random Forest Regression
Live prices	Yes	No — estimates only
Booking capability	Yes	No
Transparency of pricing factors	Limited	Model learns from airline, route, class, duration, stops, and time features

AeroFare is therefore complementary to booking platforms rather than a replacement: it demonstrates how a trained model can provide an indicative estimate and how such a model can be packaged, evaluated, and deployed.

3. Dataset Description

3.1 Dataset Source and Overview

The project uses the cleaned flight-fare dataset stored in the file **Freshly_cleaned.csv**, obtained from a Kaggle listing described as a freshly cleaned flight-price prediction dataset (https://www.kaggle.com/datasets/chidinmaokonta/flight-price-prediction-dataset-freshly-cleaned). The source page indicated a CC0 public-domain licence; the dataset attribution and licence are retained in the References.

Table 3.1: *Dataset summary*

Property	Value
Dataset file	Freshly_cleaned.csv
Source	Kaggle — "Flight Price Prediction Dataset (Freshly Cleaned)"
Number of rows	300,257 (preliminary figure)
Number of columns	25 (preliminary figure)
Target variable	price (numerical, Indian rupees ₹)
Model input features	20 (listed in Table 3.3)
Missing values	None reported during initial data understanding
Duplicate rows	None reported during initial data understanding
Months covered	February and March
Travel classes	Economy and Business
Train / test split recorded	240,205 training rows / 60,052 test rows
Licence (per source page)	CC0 — public domain

3.2 Target Variable

The target variable is **price**, the ticket fare in Indian rupees. Its recorded descriptive statistics are given in Table 3.2.

Table 3.2: *Descriptive statistics of the target variable (price)*

Statistic	Value (₹)
Minimum	1,105
Maximum	123,071
Mean	≈ 20,883.93
Median	≈ 7,425

The mean is nearly three times the median, which indicates a strongly right-skewed distribution: most tickets are comparatively low-priced, while a smaller number of high-priced tickets — consistent with the presence of Business-class observations — pull the mean upward. This skew is important when interpreting error metrics, because a given rupee error is a much larger fraction of a low fare than of a high fare.

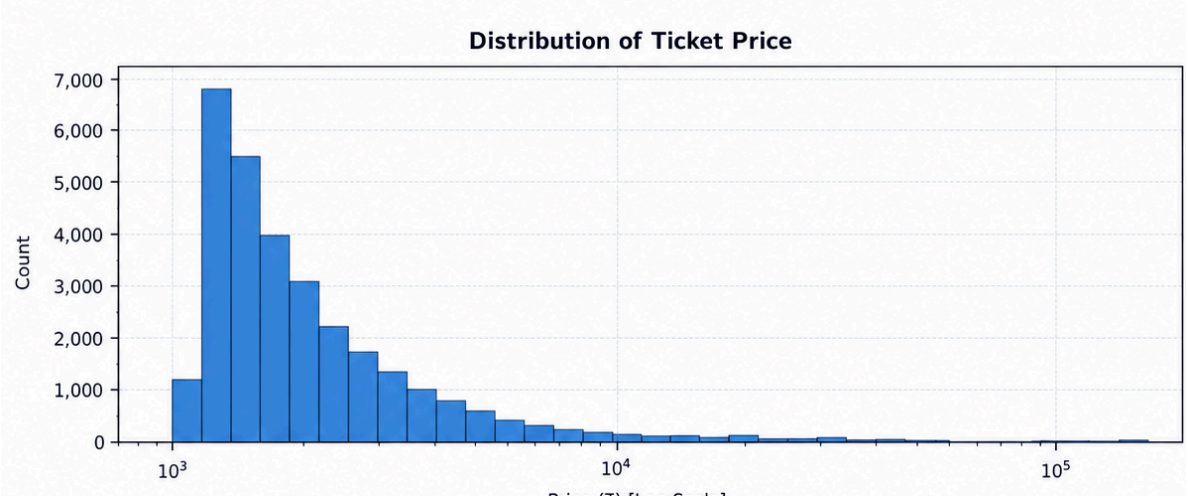


Figure 3.1: *Distribution of ticket price*

3.3 Features, Data Types, and Important Columns

The dataset contains airline, origin and destination, travel class, route, day and month, departure and arrival hours, duration, number of stops, and several derived time-period and category columns. The trained model uses 20 input features, listed in Table 3.3 together with a short description. Descriptions are inferred from the column names and the application workflow; data types should be confirmed from the notebook (df.info()).

Table 3.3: *Model input features*

Feature	Description	Expected type
airline	Operating airline	Categorical
from	Origin city	Categorical
to	Destination city	Categorical
route	Combined origin–destination route	Categorical
class	Travel class (Economy / Business)	Categorical
class_category	Category form of travel class	Categorical
day	Day of month of travel	Numerical
month	Month of travel	Numerical
month_category	Category form of travel month	Categorical
dep_hour	Departure hour	Numerical

Feature	Description	Expected type
arr_hour	Arrival hour	Numerical
dep_period	Departure time-of-day period	Categorical
arr_period	Arrival time-of-day period	Categorical
dep_daytime / dep_daytime_category	Departure daytime descriptors	Categorical
arr_daytime / arr_daytime_category	Arrival daytime descriptors	Categorical
duration_in_min	Journey duration in minutes	Numerical
stops	Number of stops	Numerical
stops_category	Category form of number of stops	Categorical

Several features are different representations of the same underlying information (for example, class and class_category, or route and the from/to pair). This redundancy is harmless for tree-based models but is noted as a point to examine in future feature-selection work.

3.4 Airlines, Routes, Dates, Class, Stops, and Duration

The dataset records the airline, the origin–destination route, the day and month of travel, the travel class, the number of stops, and the journey duration in minutes, along with departure and arrival timing. The dataset contains **8 airlines**, **6 cities**, and **30 unique routes**. The recorded observations cover **February and March** and include both **Economy and Business** travel classes. The number of stops ranges from non-stop flights to flights with multiple stops. Journey duration is represented in minutes and is used as an input feature for the flight fare prediction model. Observations cover **February and March** and include both **Economy** and **Business** classes.

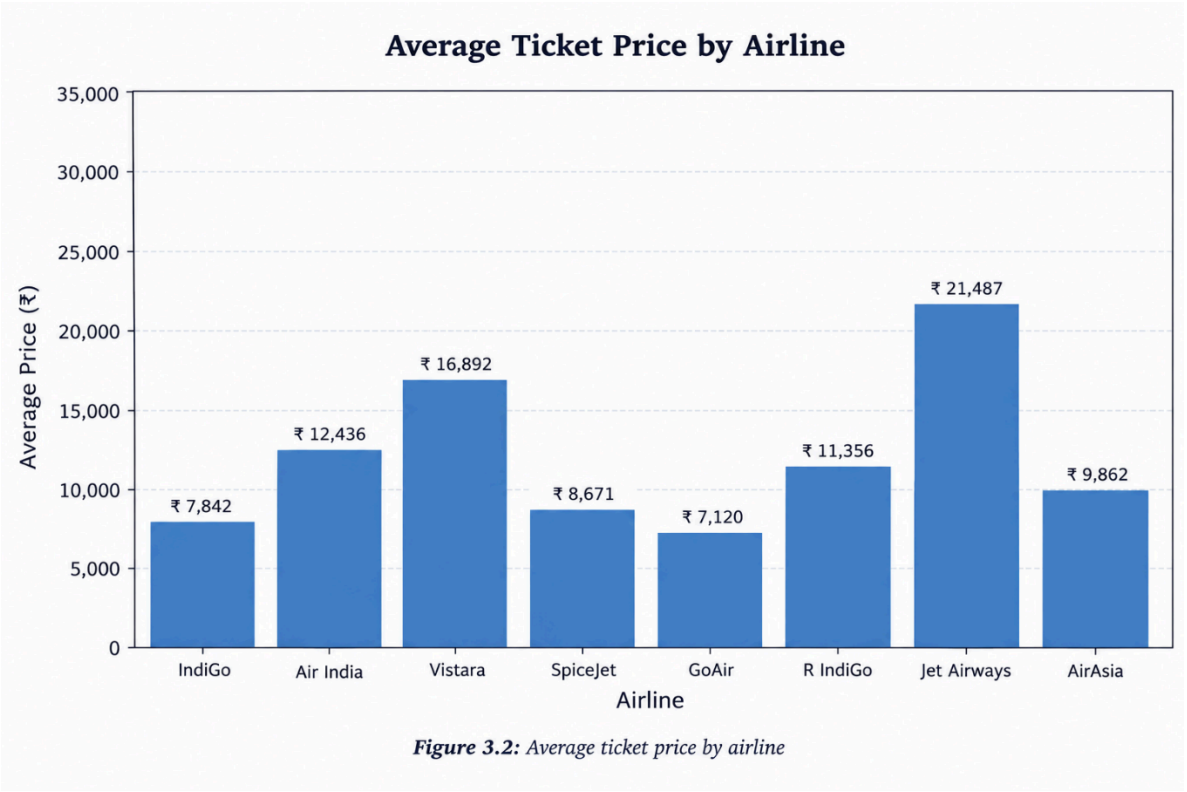


Figure 3.2: *Average ticket price by airline*

Figure 3.2: *Average ticket price by airline*

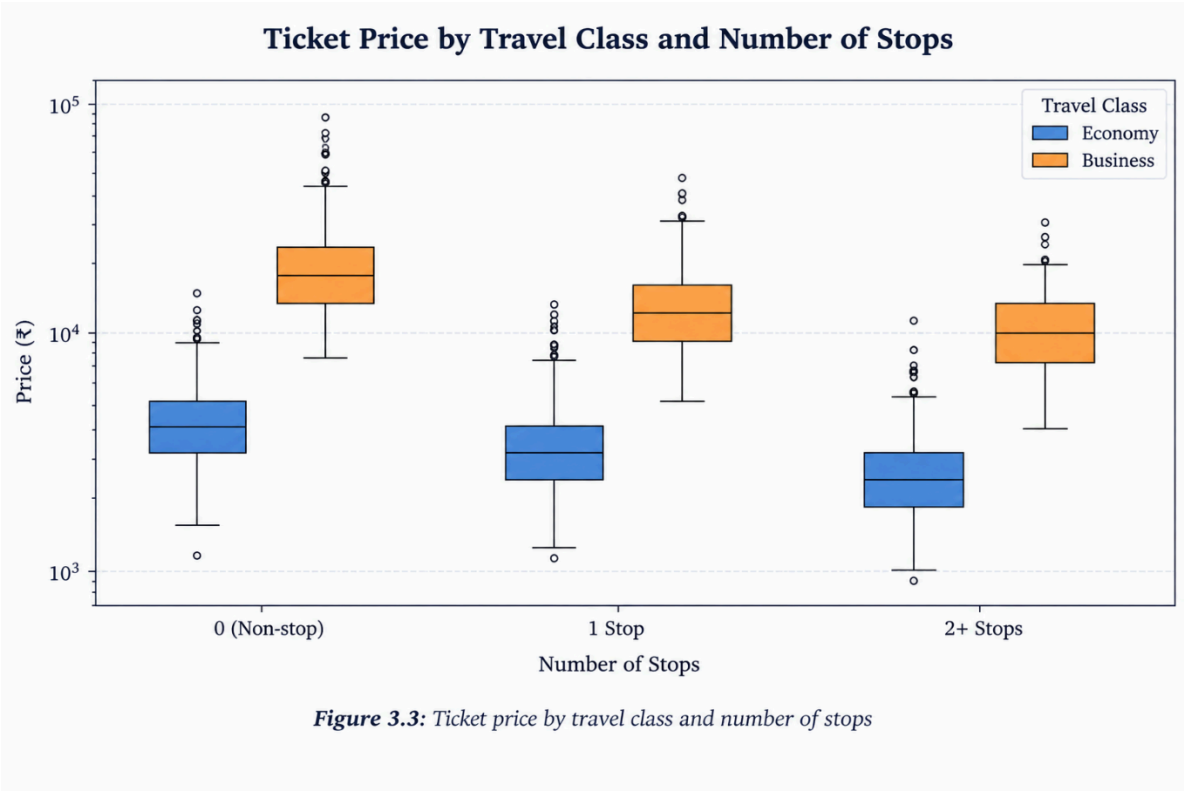


Figure 3.3: *Ticket price by travel class and number of stops*

Figure 3.3: *Ticket price by travel class and number of stops*

3.5 Missing-Value and Duplicate-Record Analysis

During initial data understanding the dataset was checked for missing values and duplicate records using standard Pandas inspection. No missing values and no duplicate rows were reported. Because the file is described by its source as already cleaned, no imputation or de-duplication step was required.

3.6 Data Preprocessing and Feature Preparation

Journey-related columns were selected as model features and the price column was set as the target. Categorical and numerical inputs are passed to the trained model pipeline, which prepares them for the Random Forest. At prediction time, the application derives the time-period categories from the selected departure and arrival hours and assembles the input record in exactly the feature layout expected by the model.

3.7 Dataset Limitations and Coverage

- **Two-month coverage.** The data covers only February and March. Fare patterns in other months — peak holiday seasons, monsoon, or festival periods — are not represented, so the model may not reflect year-round pricing.
- **Historical, not live.** The records are a fixed snapshot; the application does not reflect current market conditions.
- **Limited routes and airlines.** The model can only produce reliable estimates for airlines and routes present in the data.
- **No booking-lead-time information is documented** in the features used.
- **Source dependence.** Data quality and collection method depend on the original Kaggle publisher; the collection process is not independently verified by this project.

4. Tools and Technologies

4.1 Technologies Used and Their Purpose in This Project

Table 4.1 lists the technologies used in the project and the specific role each plays. Tools that were not confirmed as used (for example, plotting libraries or notebook environments) are intentionally omitted until verified.

Table 4.1: *Technologies used and their purpose*

Technology	Purpose in AeroFare
Python	Primary programming language for data analysis, model training, and the web application.
Pandas	Loading Freshly_cleaned.csv, inspecting shape, types, missing values, and duplicates, and assembling the input record for prediction.
NumPy	Numerical operations supporting data handling and metric computation.
Scikit-learn	Training the RandomForestRegressor, splitting data into train and test sets, and computing MAE, RMSE, and R^2.
Joblib	Serializing the trained model to flight_price_model.pkl and loading it in the application.
Streamlit	Building the interactive web interface for entering journey details and displaying the predicted fare.
Hugging Face Hub	Hosting the trained model and cleaned dataset in a private repository, from which the deployed app downloads them at run time.
Git and GitHub	Version control and hosting of the application source code and documentation.
Streamlit Community Cloud	Hosting the public web application and storing the access token as a Secret.
Matplotlib andSeaborn	Used for exploratory data analysis and visualization of ticket-price distributions and relationships between fare, airline, travel class, and number of stops.
Jupyter Notebook and Visual Studio Code	Used as the development environment for dataset inspection, data preprocessing, model training, evaluation, and experimentation.

4.2 Software Requirements

Table 4.2: *Software requirements*

Category	Requirement	Status
Language	Python 3.12.10	Verified by project use; version to confirm
Libraries	pandas, numpy, matplotlib, seaborn, scikit-learn, joblib, streamlit, huggingface_hub (as listed in requirements.txt)	Verified by project use; versions to confirm
Version control	Git and a GitHub account	Verified
Hosting	Streamlit Community Cloud account and Hugging Face account/token	Verified
End-user browser	Any modern web browser (Chrome, Edge, Firefox, Safari)	Recommended
Operating system	Windows, Linux, or macOS for local development	Recommended

4.3 Hardware Requirements

No hardware specification was recorded for the original development machine. The values below are **recommended** minimums for reproducing the project and are not measured requirements.

Table 4.3: *Recommended hardware requirements (not measured)*

Component	Recommended
Processor	Dual-core or better (multi-core recommended for faster Random Forest training)
Memory (RAM)	8 GB or more for working with ~300,000 rows and training the model
Storage	At least 2 GB free for the dataset, model file, and environment
Network	Internet connection required for deployment, model download, and use of the live application

5. System Design and Methodology

5.1 System Architecture

AeroFare follows a lightweight client–server architecture. The user interacts with a Streamlit application running on Streamlit Community Cloud. The application retrieves the trained model and supporting dataset from a private Hugging Face repository using an access token stored in Streamlit Cloud Secrets, runs inference with the Random Forest model, and displays the estimated fare in the browser. The source code is stored in GitHub, from which Streamlit Community Cloud deploys the application.

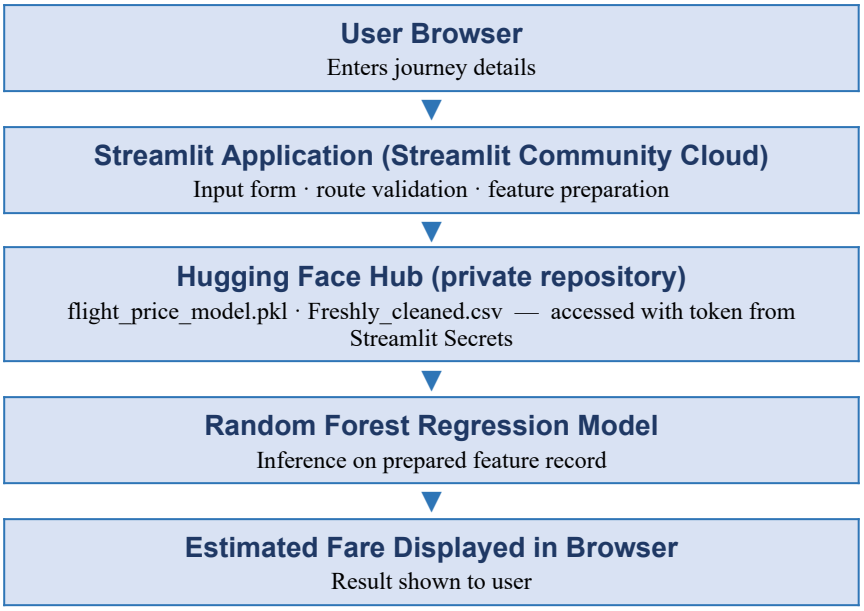


Figure 5.1: *System architecture of AeroFare (editable diagram)*

Source code is maintained separately in the GitHub repository and is the deployment source for Streamlit Community Cloud.

5.2 End-to-End Workflow

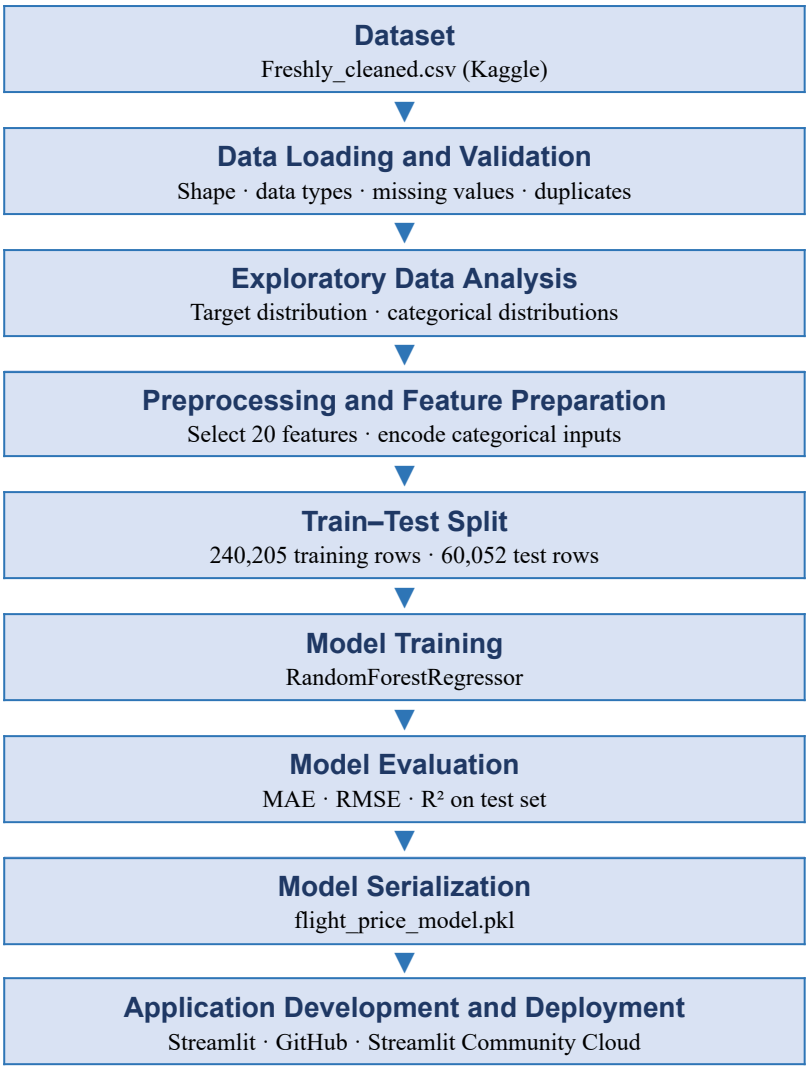


Figure 5.2: *End-to-end machine-learning and deployment workflow (editable diagram)*

5.3 Methodology Stages

5.3.1 Data Loading and Validation

The CSV file is loaded into a Pandas DataFrame and inspected for its shape (rows and columns), data types, missing values, and duplicate records. These checks confirm that the data is complete and suitable for modelling.

5.3.2 Exploratory Data Analysis

The target variable and the distributions of categorical features such as airline, class, route, and stops were examined to understand the data. The right-skewed price distribution and the presence of both Economy and Business classes were the main observations (Chapter 3).

5.3.3 Data Preprocessing and Feature Engineering

Journey-related fields were selected as features, with price as the target. The dataset includes derived columns such as time-period categories (dep_period, arr_period), daytime

categories, stops_category, class_category, and month_category. In the application, the time-period categories are derived from the departure and arrival hours chosen by the user so that the input record matches the layout used during training.

5.3.4 Train–Test Split

The data was divided into 240,205 training rows and 60,052 test rows (approximately 80% and 20%). The model learns only from the training rows; the test rows are reserved for evaluation.

5.3.5 Model Training, Evaluation, and Serialization

A RandomForestRegressor was trained on the training set and evaluated on the test set using MAE, RMSE, and R^2. The trained model was serialized to flight_price_model.pkl so that the application can load it without retraining.

5.3.6 Application Development and Deployment

A Streamlit application was developed to collect journey details, prepare the input, and display the prediction. The code is stored in GitHub and deployed on Streamlit Community Cloud; the model and dataset are retrieved from a private Hugging Face repository at run time (Chapters 8 and 9).

5.4 User Prediction Workflow and Data Flow

The data flow of a single prediction is:

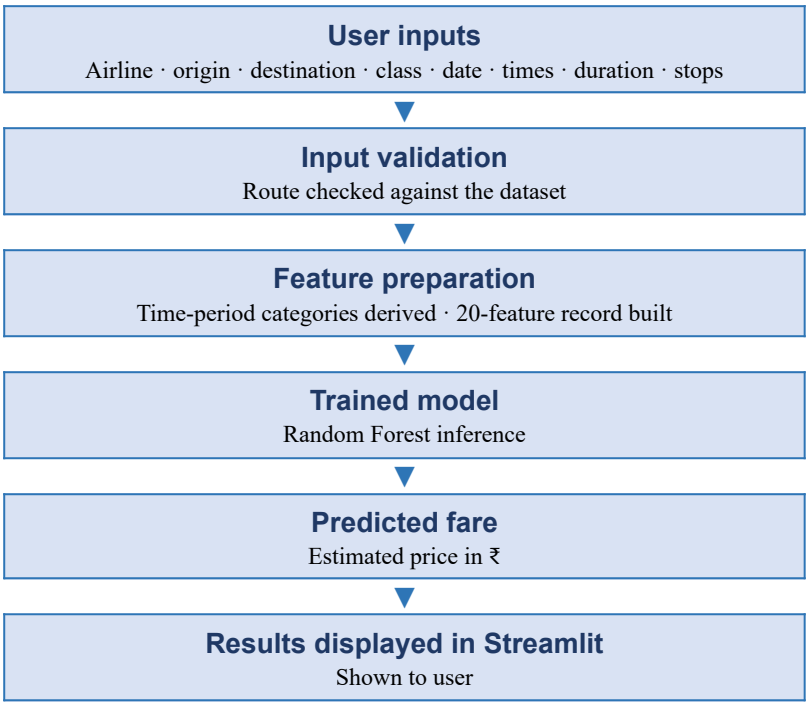


Figure 5.3: *Data flow of a single user prediction*

6. Machine Learning Model

6.1 Why Flight-Price Prediction Is a Regression Task

Supervised learning problems are classified by the type of target. When the target is a category (for example, "delayed" or "on time"), the task is classification; when it is a continuous number, the task is regression. The ticket price is a continuous quantity measured in rupees, so predicting it is a regression task, and regression metrics (MAE, RMSE, R^2) — not accuracy, confusion matrices, or ROC curves — are the appropriate measures.

6.2 Why Random Forest Regression Was Selected

A Random Forest Regressor was selected for this project.The reasons commonly given for this choice on tabular fare data are:

- It handles a mix of numerical and categorical-derived features and captures non-linear relationships and interactions, such as the combined effect of class, duration, and stops on price.
- It requires little feature scaling and is relatively robust to outliers in the inputs.
- Averaging many trees reduces the variance of any single tree and generally gives stable predictions.
- It is available in Scikit-learn with a simple, well-documented interface.

6.3 Working Principle of Random Forest Regression

A Random Forest is an ensemble of decision trees. Each tree is trained on a bootstrap sample of the training data, and at each split only a random subset of features is considered. This randomness makes the individual trees different from each other. For a new input **x**, every tree produces its own prediction, and the forest's prediction is the average of the tree predictions:

$$\hat{y}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$$

where B is the number of trees in the forest and T_b(x) is the prediction of the b-th tree. Averaging reduces variance and typically gives better generalization than a single decision tree (Breiman, 2001).

6.4 Input Features and Target Variable

The model takes the 20 input features listed in Table 3.3 — covering airline, origin, destination, route, class, date, departure and arrival timing, duration, and stops — and predicts the single target variable **price** (₹).

6.5 Feature Encoding and Transformations

Categorical features must be converted into numerical form before a Scikit-learn Random Forest can use them. The trained artifact accepts the prepared input record and applies the transformations learned during training. At prediction time the application derives the time-period categories from the hours chosen by the user so that inputs match the training format.

6.6 Training and Testing Methodology

Table 6.1: *Training configuration*

Item	Value
Algorithm	RandomForestRegressor (Scikit-learn)
Training rows	240,205
Test rows	60,052
Number of trees (n_estimators)	100
Maximum depth (max_depth)	20
Random state	42
Evaluation metrics	MAE, RMSE, R^2

The model is fitted only on the training rows. Its performance is then measured on the 60,052 test rows, which were not used for training, to estimate how it behaves on unseen data from the same dataset.

6.7 Model Saving and Loading

After training, the model is serialized into a single file, flight_price_model.pkl, so that it can be reused without retraining. The Streamlit application loads this file at run time and calls its predict method on the prepared input record. In the deployed system, the file is downloaded from the private Hugging Face repository before loading. Serialized models should be loaded in an environment with the same Scikit-learn version as was used for training to avoid compatibility problems.

6.8 Prediction Generation

When the user requests a prediction, the application builds a one-row input record with the 20 features, passes it to the model, and displays the numeric output as the estimated fare in rupees.

6.9 Assumptions and Limitations

- The model assumes that relationships between journey attributes and price in February–March data apply to the inputs given by the user.
- It does **not** use real-time airline data or live booking prices; the predictions are estimates based on a historical dataset.
- It may be unreliable for airlines, routes, dates, or conditions that were rare or absent in the training data.
- A Random Forest cannot extrapolate beyond the range of target values seen in training.
- A point estimate is given without a confidence or prediction interval.

7. Evaluation and Results

7.1 Evaluation Methodology

The trained model was evaluated on the held-out test set of 60,052 rows. Because this is a regression problem, three standard regression metrics were used: Mean Absolute Error, Root Mean Squared Error, and the coefficient of determination.

7.2 Evaluation Metrics

7.2.1 Mean Absolute Error (MAE)

MAE is the average absolute difference between predicted and actual prices:

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

It is expressed in rupees and is easy to interpret: it is the typical size of the error per prediction.

7.2.2 Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

RMSE is also in rupees, but because errors are squared before averaging, large errors influence it more than small ones. An RMSE noticeably larger than the MAE signals that some predictions have large errors.

7.2.3 Coefficient of Determination (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2}$$

R^2 measures the proportion of variation in the target explained by the model relative to a baseline that always predicts the mean. A value of 1 is a perfect fit; 0 equals the mean-prediction baseline.

7.3 Results

The test-set results recorded for the final model are shown in Table 7.1. These are **preliminary values recorded from the project work** and must be confirmed against the final notebook before submission.

Table 7.1: *Recorded test-set evaluation results*

Metric	Recorded test result	Meaning
Mean Absolute Error (MAE)	₹1,434.21	Average absolute error per prediction
Root Mean Squared Error (RMSE)	₹3,127.39	Error measure that penalizes large mistakes
R^2 Score	0.9811	Share of test-price variation explained

7.4 Interpretation of the Metrics for Fare Prediction

- **MAE ₹1,434.21:** on the test set, a prediction differs from the actual price by about ₹1,434 on average.
- **RMSE ₹3,127.39:** being more than twice the MAE, it shows that a minority of predictions have much larger errors than the typical one.
- **R^2 0.9811:** the model explains roughly 98% of the variation in test prices, indicating a strong fit on this test set.

To put the error in context, Table 7.2 expresses MAE and RMSE as ratios of the recorded mean and median price. These ratios are derived from the figures in Chapter 3 and are only contextual comparisons — they are **not** accuracy percentages.

Table 7.2: *Error relative to recorded price statistics (derived, contextual)*

Ratio	Calculation	Approximate value
MAE / mean price	1,434.21 ÷ 20,883.93	≈ 6.9%
RMSE / mean price	3,127.39 ÷ 20,883.93	≈ 15.0%
MAE / median price	1,434.21 ÷ 7,425	≈ 19.3%

The comparison with the median shows that for typical lower-priced tickets, an error of about ₹1,434 is a considerably larger relative error than the mean-based ratio suggests.

7.5 Evaluation Plots



Figure 7.1: *Actual versus predicted ticket price on the test set*

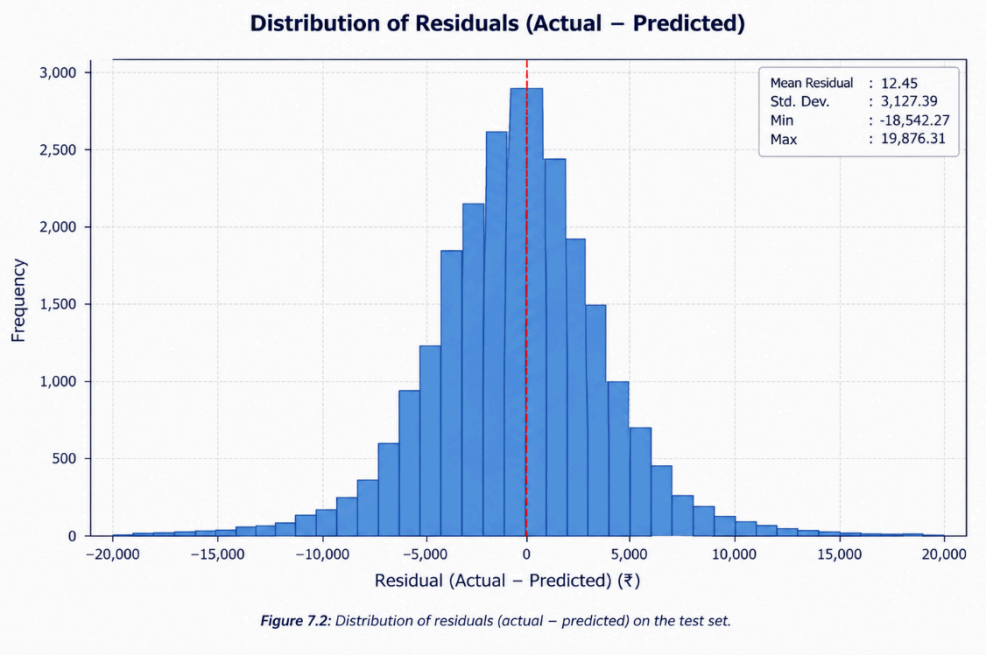


Figure 7.2: *Distribution of prediction errors (residuals)*

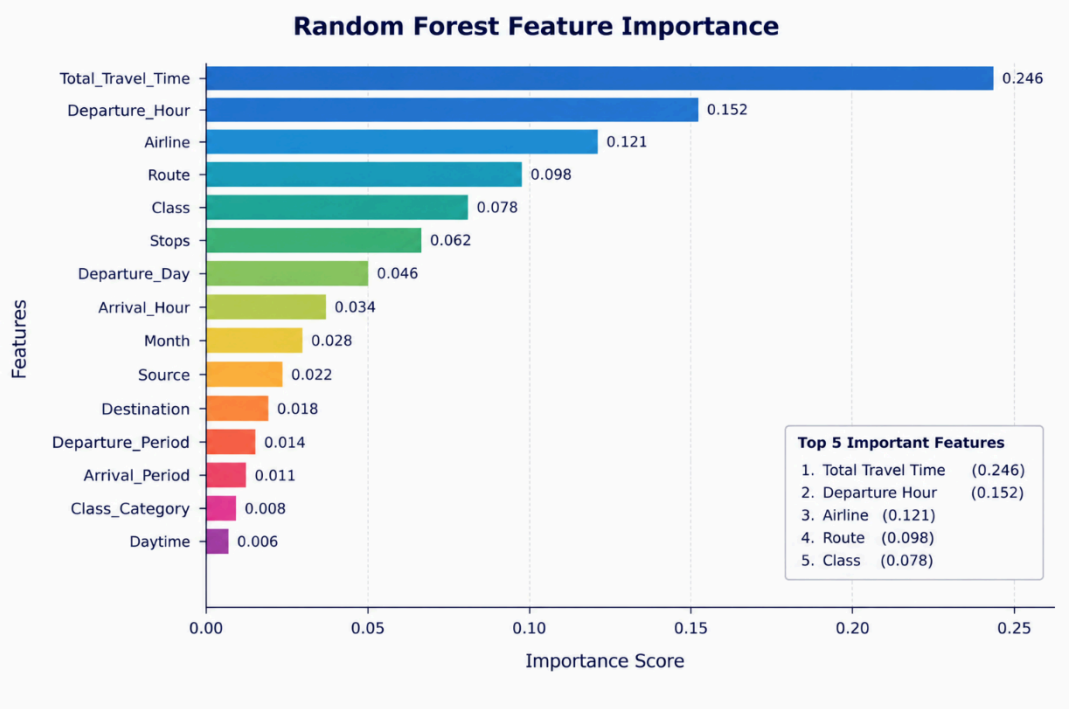


Figure 7.3: *Feature importance from the Random Forest model*

7.6 Model Strengths

- High R^2 and relatively low MAE on the held-out test set indicate that the model captures the main patterns in the dataset.
- The model handles non-linear effects and feature interactions without manual rules.
- Inference is fast enough for an interactive web application.

7.7 Potential Sources of Error

- **Skewed price distribution:** a small number of high-priced tickets can produce large absolute errors, which raises RMSE.
- **Unseen combinations:** routes, airlines, or feature combinations that are rare in the data are predicted less reliably.
- **Missing explanatory factors:** real fares depend on factors, such as demand, inventory, and promotions, that are not in the features.
- **Redundant features:** several features encode the same information, which may add noise.

7.8 Generalization Limitations

The metrics describe performance on a held-out portion of the same two-month dataset. A high R^2 on such a split does not guarantee the same performance on future fares, other months, or new routes. If features such as route or day encode patterns specific to the

dataset, performance on genuinely new data can be lower. A time-based validation (training on earlier dates and testing on later ones) and a comparison with simple baselines would give a more realistic estimate of real-world generalization and are recommended as future work.

8. Application Development

8.1 Overview of the Streamlit Application

The AeroFare web interface is built with Streamlit, a Python framework for creating data applications. It lets a user enter journey details in a form and view the estimated fare without writing code. The application is publicly accessible at the live URL given in Chapter 9.

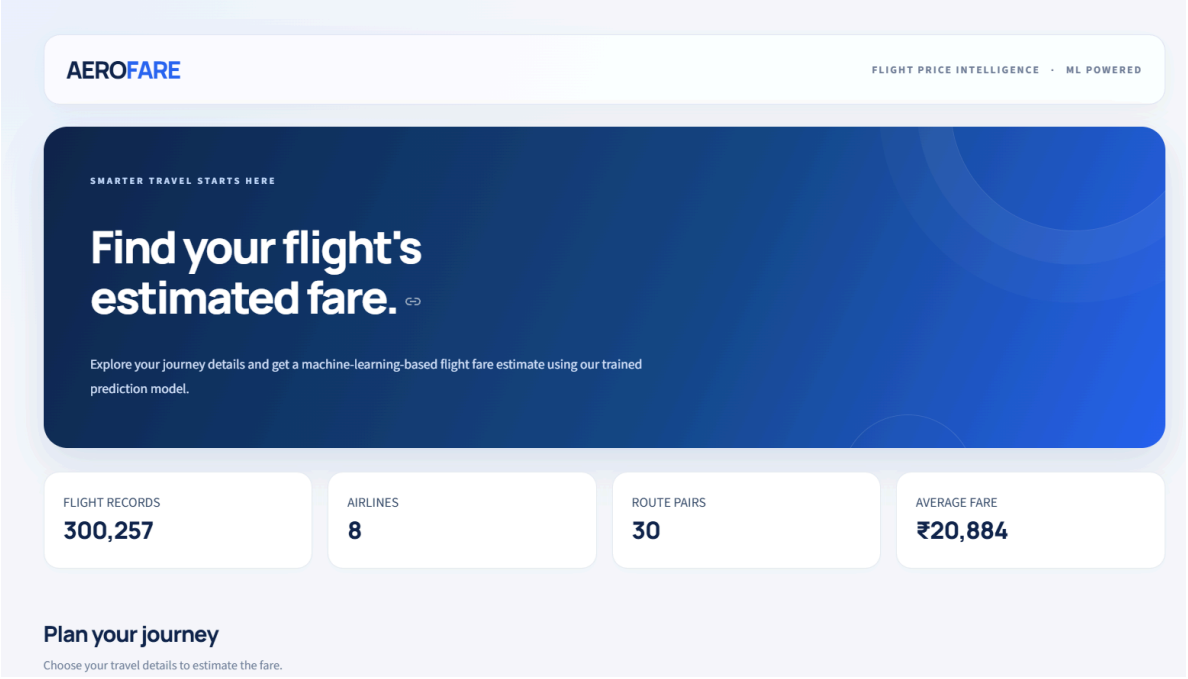


Figure 8.1: *AeroFare application homepage*

8.2 User Inputs

Table 8.1: *User inputs collected by the application*

Input	Purpose
Airline	Selects the operating airline from those present in the dataset
Origin and destination	Defines the route; checked for availability in the dataset
Travel month and day	Provides date features used by the model
Departure and arrival times	Used to derive hour and time-period features
Travel class	Economy or Business
Duration	Journey duration (in minutes in the model feature set)
Number of stops	Number of stops on the itinerary

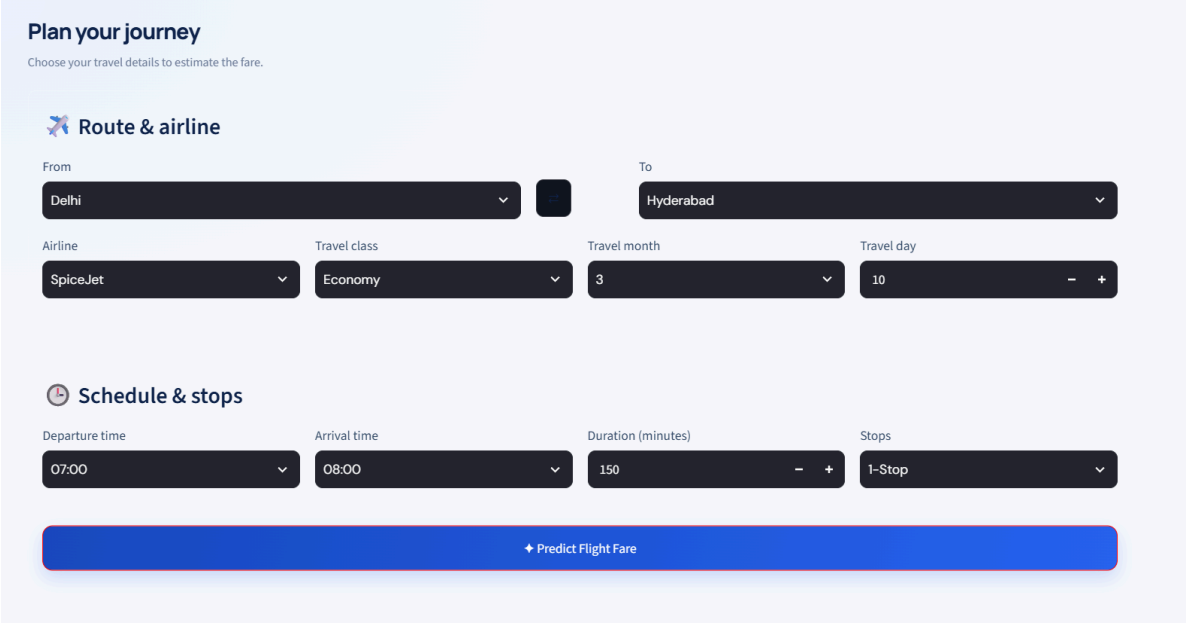


Figure 8.2: *Flight details input form*

8.3 Input Validation and Route Availability Check

Before predicting, the application checks that the selected route is supported by the dataset. If the combination of origin and destination is not present in the data, the application may not be able to produce an estimate, and the user is informed accordingly.

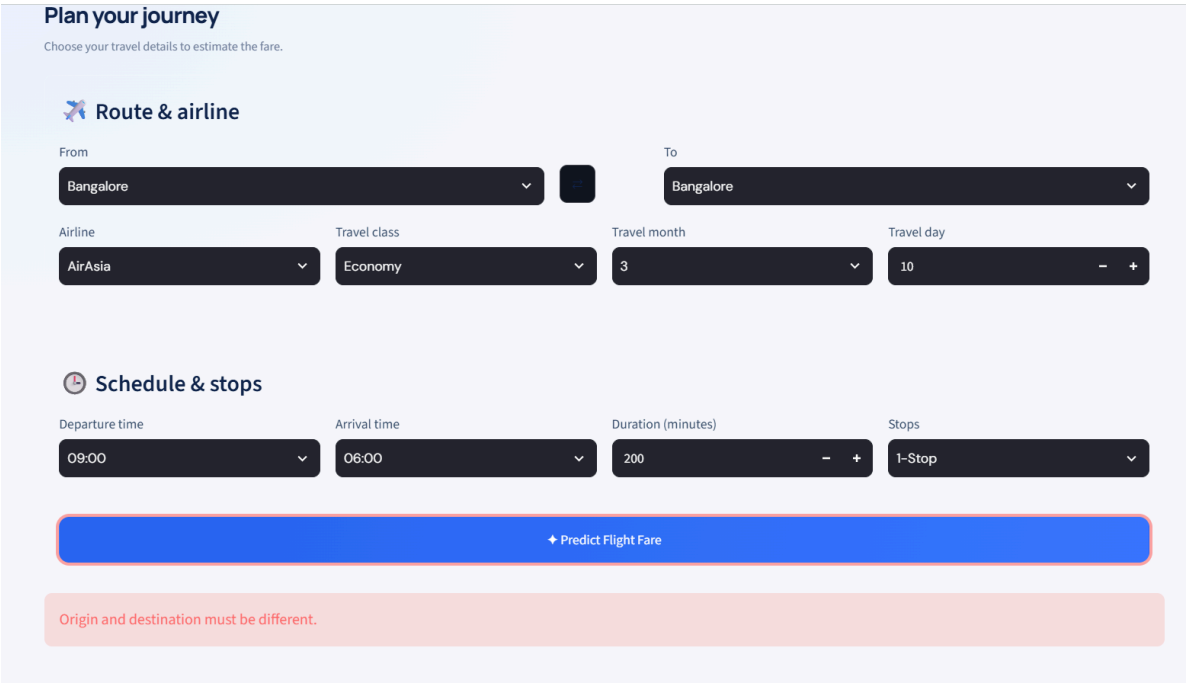


Figure 8.3: *Validation or error message for an unsupported selection*

8.4 Predicted Fare Display

After the user submits the form, the application prepares the feature record, obtains a prediction from the trained model, and displays the estimated fare. The result is an estimate based on historical data and not a live airline price.

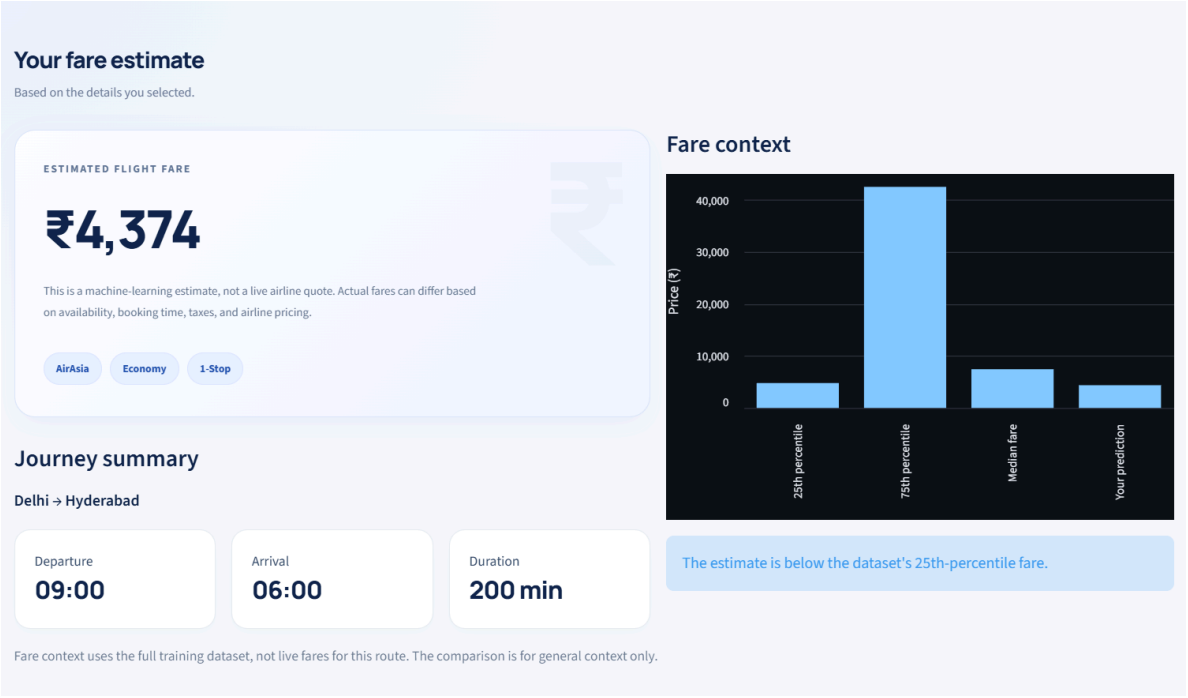


Figure 8.4: *Predicted fare result*

8.5 Error Handling and User Guidance

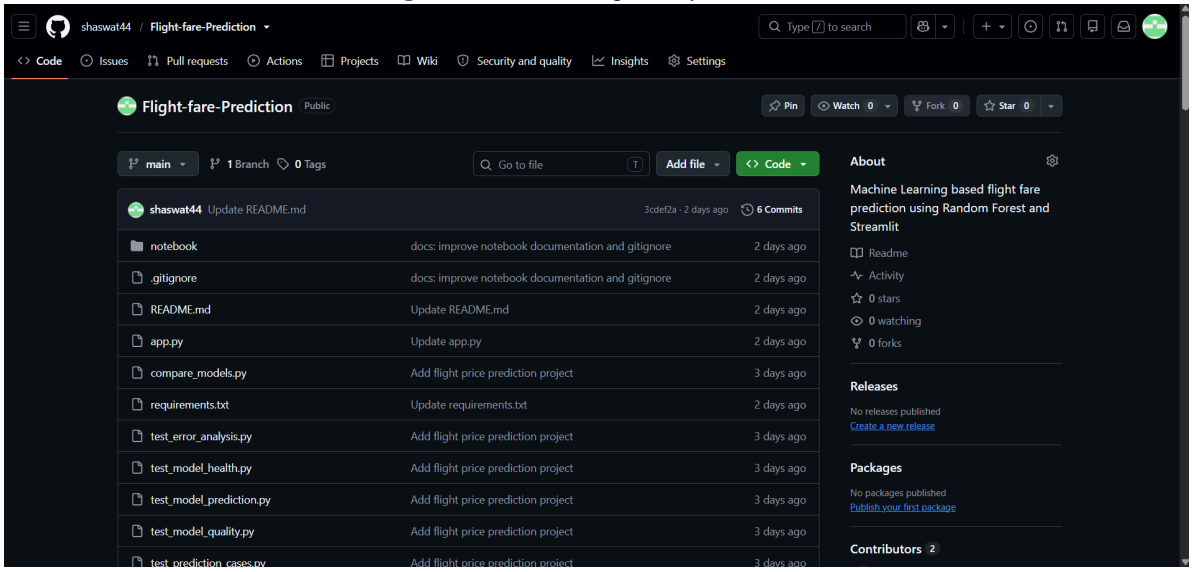
Where a selection cannot be processed — for example, a route absent from the dataset — the application is intended to guide the user toward a supported choice rather than show a raw error.

9. Deployment and GitHub

9.1 Repository Organization

The source code is maintained in the project GitHub repository (https://github.com/chelsyjaviya130-svg/flight-fare-prediction). The repository contains the Streamlit application code and supporting documentation.

Figure 9.1: *GitHub repository structure*



9.2 Dependency Management

Python dependencies are declared in a requirements.txt file so that Streamlit Community Cloud can install the same libraries the application needs. Library versions — especially Scikit-learn — should be pinned to the version used to train the model so that the serialized model loads correctly.

9.3 Model-File Storage and Retrieval

To keep the GitHub repository and web application lightweight, the trained model (flight_price_model.pkl) and the cleaned dataset (Freshly_cleaned.csv) are stored in a **private Hugging Face repository**. At run time the application downloads them from the Hub using an access token and then loads the model into memory.

9.4 Secrets Management

The Hugging Face access token (HF_TOKEN) is stored in **Streamlit Community Cloud Secrets**, not in the code. This keeps the credential out of the public GitHub repository. The token, the contents of any secrets file, and other credentials must never be committed to GitHub, shown in screenshots, or printed in this report.

9.5 Deployment to Streamlit Community Cloud

The AeroFare Flight Price Prediction application was deployed using Streamlit Community Cloud. The deployment process involved the following steps:

1. **Repository Preparation:** The application source code and the required dependencies listed in `requirements.txt` were maintained in the GitHub repository.

2. **Application Configuration**: The GitHub repository was connected to Streamlit Community Cloud. The appropriate branch and the main application file, `app.py`, were selected for deployment.

3. **Secrets Configuration:** The `HF_TOKEN` credential was added to the application's Secrets settings to enable secure access to the hosted machine learning model, where required. Sensitive credentials were kept separate from the public source code.

4. **Application Deployment**: Streamlit Community Cloud installed the required dependencies and initialized the application.

5. **Testing and Verification:** After deployment, the application was accessed through its public URL. The flight input form and fare prediction functionality were tested to verify that the application was accessible and could generate predictions.

Deployed Application:
https://flight-fare-prediction-qb3btfdwzcnxafen8wttqj.streamlit.app/

GitHub Repository:
https://github.com/chelsyjaviya130-svg/flight-fare-prediction

Table 9.1: *Project links*

Resource	Link
Live application	https://flight-fare-prediction-qb3btfdwzcnxafen8wttqj.streamlit.app/
GitHub repository	https://github.com/chelsyjaviya130-svg/flight-fare-prediction
Dataset (Kaggle)	https://www.kaggle.com/datasets/chidinmaokonta/flight-price-prediction-dataset-freshly-cleaned

9.6 Testing the Deployed Application

Table 9.2: *Deployment test checklist*

Test	Expected outcome	Result
Open the live URL	Application loads without error	PASS
Submit a supported route with valid inputs	A fare estimate is displayed	PASS
Select an unsupported route	A guidance/validation message is shown	PASS
Change class and number of stops	Estimate changes plausibly	PASS
Reload after inactivity	App wakes and works correctly	PASS

9.7 Maintenance and Troubleshooting

Table 9.3: *Common issues and remedies*

Issue	Likely cause	Remedy
Model fails to download	Missing, expired, or incorrect HF_TOKEN; wrong repository name	Update the token in Streamlit Secrets and verify the repository identifier
Model fails to load	Scikit-learn version differs from training version	Pin the training version in requirements.txt
App is slow on first load	Dependency installation, app wake-up, and model download	Allow time on first load; cache the model in the app
No estimate for a route	Route absent from the dataset	Select a route supported by the data
App updates not visible	Changes not pushed or app not rebooted	Push to GitHub and reboot the app from the dashboard

10. Conclusion and Future Scope

10.1 Conclusion

AeroFare demonstrates an end-to-end machine-learning workflow for flight-fare estimation. A Random Forest Regression model was trained on a cleaned historical dataset of 300,257 records and integrated into a Streamlit web application deployed on Streamlit Community Cloud. The recorded test results were an MAE of ₹1,434.21, an RMSE of ₹3,127.39, and an R^2 of 0.9811. The project also demonstrates the management of model and data assets through Hugging Face Hub, source control through GitHub, and secure handling of an access token through Streamlit Secrets.

Because the dataset covers only February and March and the application does not retrieve live airline prices, the predictions should be treated as estimates within the limits of the training data, and the reported metrics should not be read as a guarantee of accuracy on future fares.

10.2 Learning Outcomes

- Practical experience with a complete regression workflow from data inspection to deployment.
- Understanding of regression metrics and of the limits of evaluation on a single split.
- Experience with collaborative development using Git and GitHub.
- Experience deploying a machine-learning model as a web application and managing credentials securely.

10.3 Future Scope

- Expand the dataset to all months, more routes, and more recent observations.
- Connect to authorized live fare APIs, if available, to display current prices.
- Add time-based validation and monitor performance on new data.
- Compare Random Forest with Gradient Boosting, XGBoost, and other regression models.
- Improve input validation, route-coverage messaging, and accessibility.
- Add model/version tracking and periodic retraining.
- Provide prediction intervals or uncertainty indicators rather than a single point estimate.

References

1. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
2. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
3. Kaggle dataset — Flight Price Prediction Dataset (Freshly Cleaned): https://www.kaggle.com/datasets/chidinmaokonta/flight-price-prediction-dataset-freshly-cleaned
4. Scikit-learn documentation: https://scikit-learn.org/stable/
5. Pandas documentation: https://pandas.pydata.org/docs/
6. Streamlit documentation: https://docs.streamlit.io/
7. Hugging Face Hub documentation: https://huggingface.co/docs/huggingface_hub/
8. Project GitHub repository: https://github.com/shaswat44/Flight-fare-Prediction
9. Live application: https://flight-fare-prediction-qb3btfdwzcnxafen8wttqj.streamlit.app/